Stealing Graph Neural Network Models

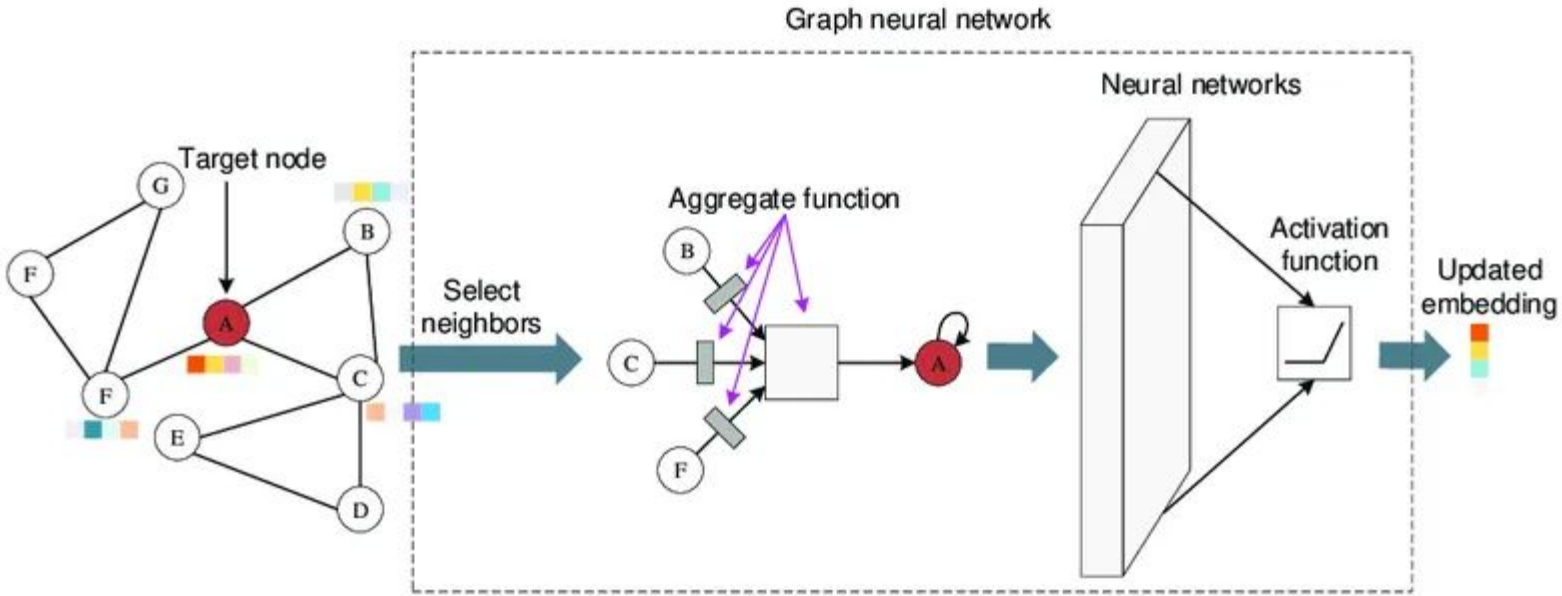
Marcin Podhajski

Stealing Graph Neural Network Models

- What is model stealing?
- Why is it a threat?
- Stealing Graph Neural Networks

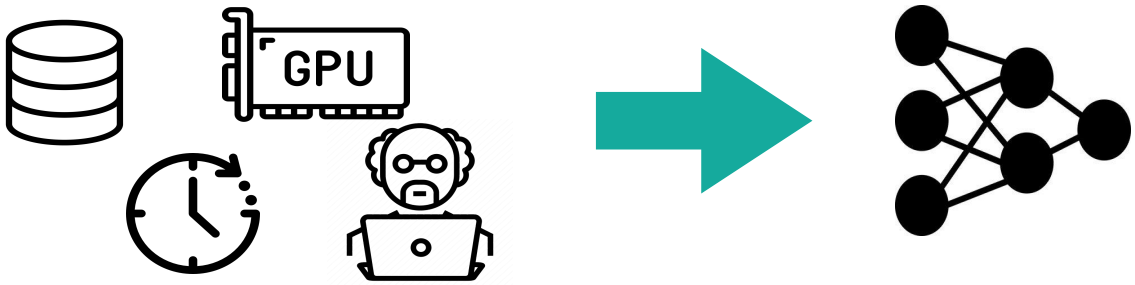
Graph neural networks

- **Graph neural networks** (GNNs) are a powerful tool for processing graph-structured data.
- This work focuses on **node-level tasks** in the **inductive setting**.
- Inductive GNNs are crucial as they can generalize to unseen graph data without depending on predefined graph structures.

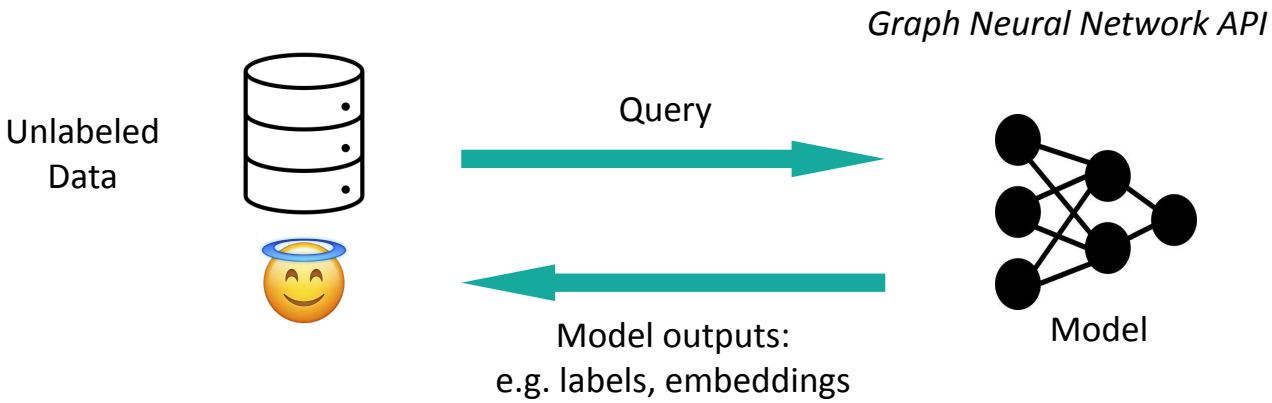

Graph neural network
Target node
G
B
F
A
F
C
E
D
Select
neighbors
Aggregate function
B
C
A
F
Neural networks
Activation
function
Updated
embedding

Graph neural networks

- Training GNNs requires data, time and compute
- Such models can be made publicly available behind an **model API**
- As GNNs grow in popularity, they are becoming an attractive target for **model stealing attacks**.

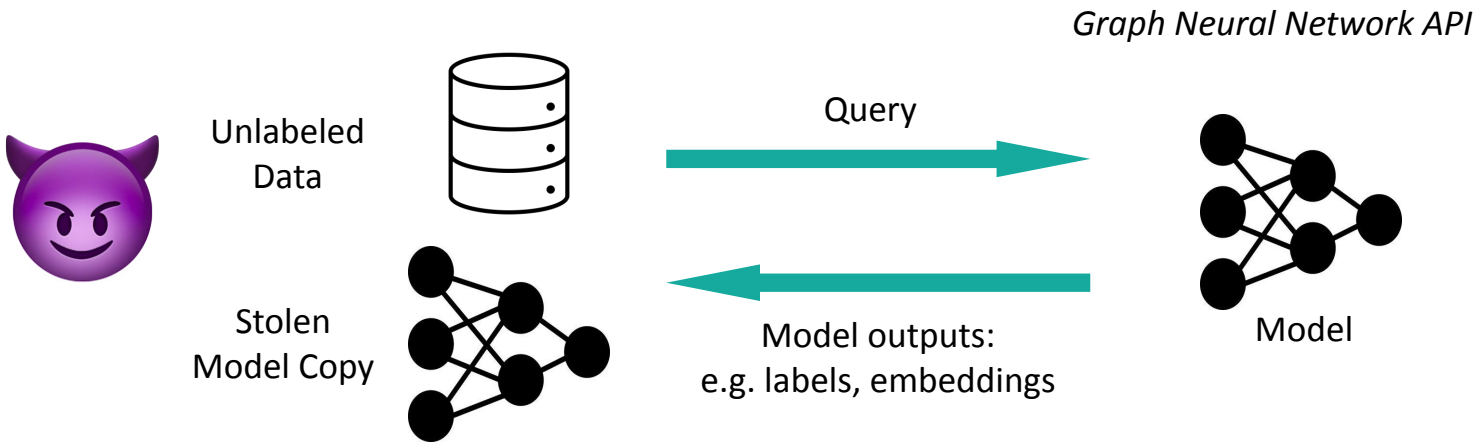


Model stealing



Legitimate users send graph data to the GNN model behind API to get its outputs
The user uses the model API to solve a particular task, e.g. node classification

Model stealing



An attacker uses outputs from the GNN model API to train a local model copy

Model stealing

- **Model stealing is a serious threat**: Attackers can replicate models by exploiting access to their outputs.
- **Less resource-intensive than training**: Stealing models requires much fewer resources than building and training one from scratch.
- Adversary's goals:
  - **theft**: create a surrogate model that replicates the performance of the target model
  - **reconnaissance**: develop a surrogate model MS that approximates across all possible inputs which poses a serious security risk

Real-world threats

- Monetization Strategy: Companies sell access to their models rather than sharing the actual internals.
- Black-Box Distribution: Models are provided as executables (.exe), preventing direct inspection.
- Intellectual Property Protection: Companies aim to safeguard proprietary methods and data.
- Security Implications: This setup invites model extraction attacks, where adversaries attempt to recreate the model by probing it.

GNN model stealing

- Most model stealing research focuses on image and text models.
- **Attacks on GNNs are poorly studied**, despite the growing importance of these networks.

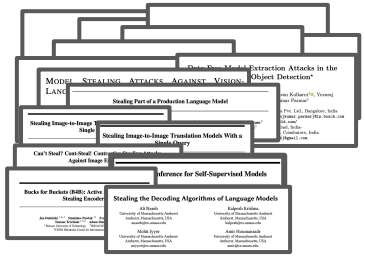
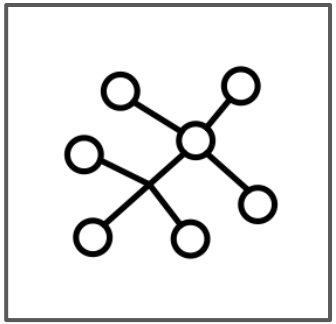
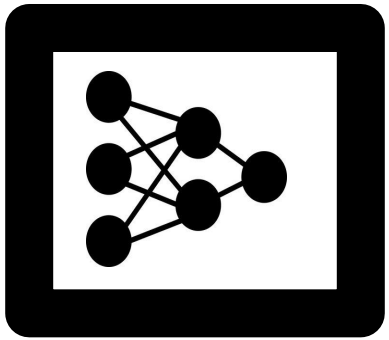


Image / Language
Model Stealing



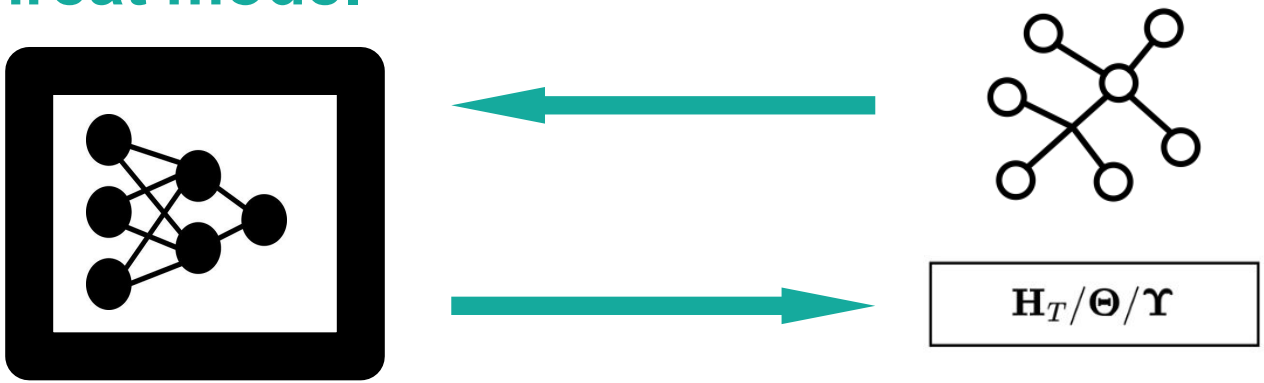
GNN
Model Stealing

Threat model



Black-box scenario:
The adversary has no knowledge of the GNN model's internal details, such as its parameters or architecture.

Threat model



Query access:
The adversary can interact with the target model by sending queries and receiving outputs (e.g., **node embeddings, predicted probabilities, t-SNE projections**).

Threat model

Query data

Target GNN training data

Query graph from the same distribution:

The adversary can access a query graph drawn from the same dataset used to train the target GNN model.

Experimental evaluation and results

- Attacks were evaluated on:
  - six datasets: DBLP, Pubmed, Citeseer, Coauthor Physics, ACM, Amazon.
  - three GNN models: GIN, GAT, GraphSAGE.

- We evaluate our method in terms of:
  - **accuracy and F1 score** - how good is the stolen model?
  - **fidelity** - how similar is the stolen model to the target model?
  - **the required number of queries** - how efficient is our method?

2 limitations on model stealing

An adversary attempting to steal a model can face 2 main limitations:

- lack of access to the data
- lack of access to the target model

Limitation on data

Model Extraction Depends on Available Data

- Attackers need sufficient queries to reconstruct a model.
- More diverse and representative data leads to a more accurate stolen model.
- Limited or biased data results in incomplete or inaccurate reconstructions.

Limitation on the access to the model

- Access Constraints Affect Model Extraction

- Attackers rely on querying the model to gather information.
- Limited access makes it harder to reconstruct the model accurately.

- Common Restrictions

1. Query Budget – APIs often limit the number of queries per user or per day.
2. Rate Limiting – Excessive queries trigger temporary or permanent bans.
3. Access Controls – Authentication, paywalls, or usage tracking restrict unauthorized users.

1st Attack Strategy: Generating More Data

What If the Attacker Has Partial Training Data?

- If the attacker possesses some original training data or data from the same distribution, model extraction becomes easier.
- They can use this data to generate meaningful queries, improving the quality of the stolen model.

Key Assumptions

- The attacker can query the model freely with this known data.
- Fewer queries are still preferred to avoid detection and stay within query limits.
- We assume that the target returns encoder embeddings or something similar

Encoder + head

- Encoder

- Extracts meaningful features from raw input (e.g., images, text, molecules).
- Often consists of convolutional layers, transformers, or graph networks.
- Acts as a bottleneck, reducing raw data to a structured representation.

- Head

- Takes the encoded representation and maps it to the final output.
- Can be a classification layer, regression layer, or another task-specific module.
- Typically consists of fully connected layers.

Encoder outputs

- Encoders Produce Embeddings

  - Encoders transform raw input (e.g., images, text, molecules) into compact, structured representations.
  - These embeddings capture meaningful features while discarding irrelevant details.
  - Example: A face recognition model encodes faces into unique vector representations.

- Why Embeddings Are Useful

  - They allow efficient storage, retrieval, and transfer of knowledge.
  - Can be shared between models for transfer learning or interoperability.
  - Often used in APIs for feature extraction without exposing full models.

Head outputs

- The Head Translates Embeddings to Outputs

- The head of a neural network takes embeddings and produces the final prediction (e.g., class labels, scores).
- Typically consists of fully connected layers trained for a specific task.
- Example: A medical AI model's head converts patient embeddings into disease predictions.

What is Self-supervised learning?

What is SSL?

- A training approach where models learn without labeled data.
- The model generates its own pseudo-labels from the data.
- Bridges the gap between unsupervised and supervised learning.

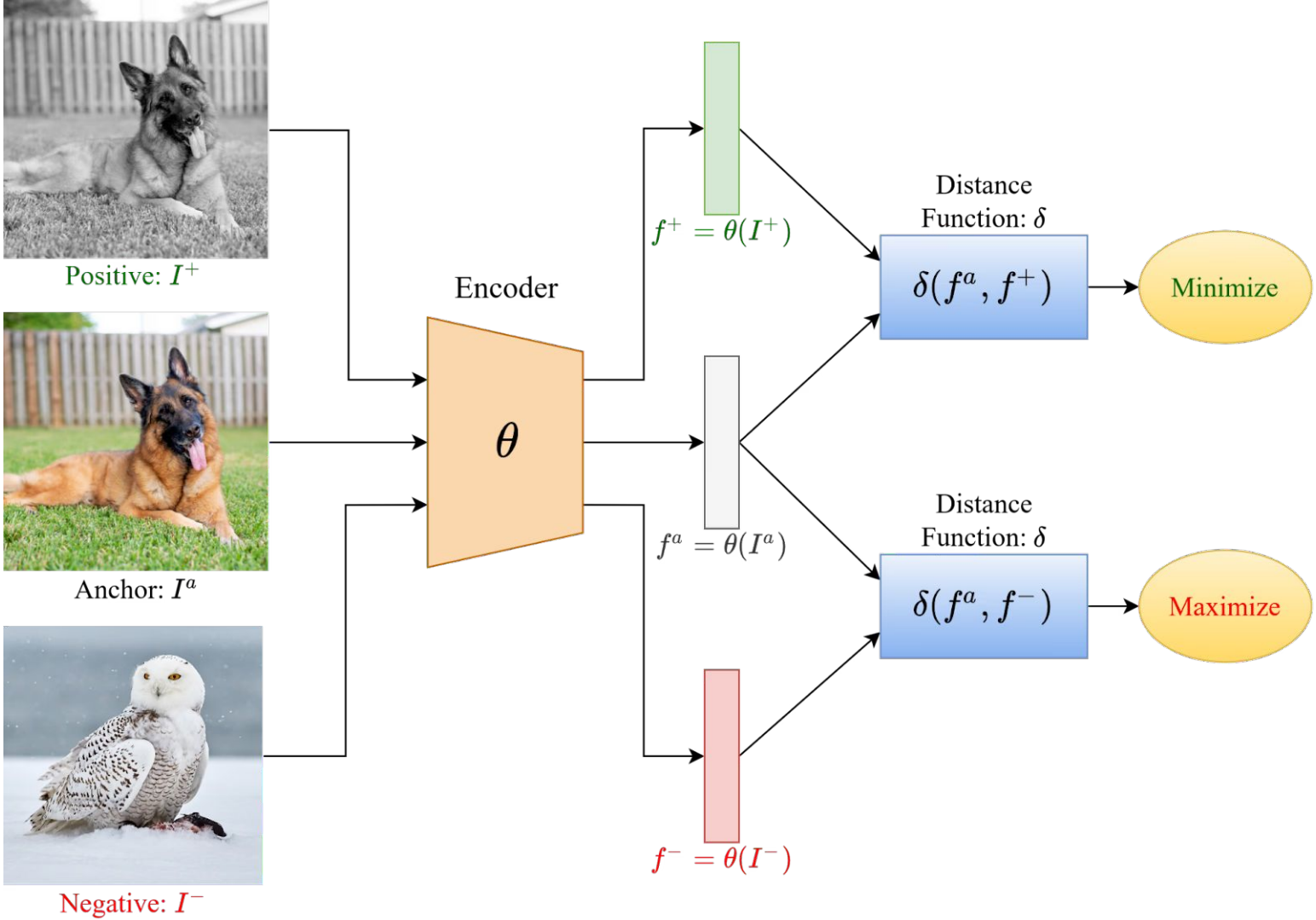
- How It Works

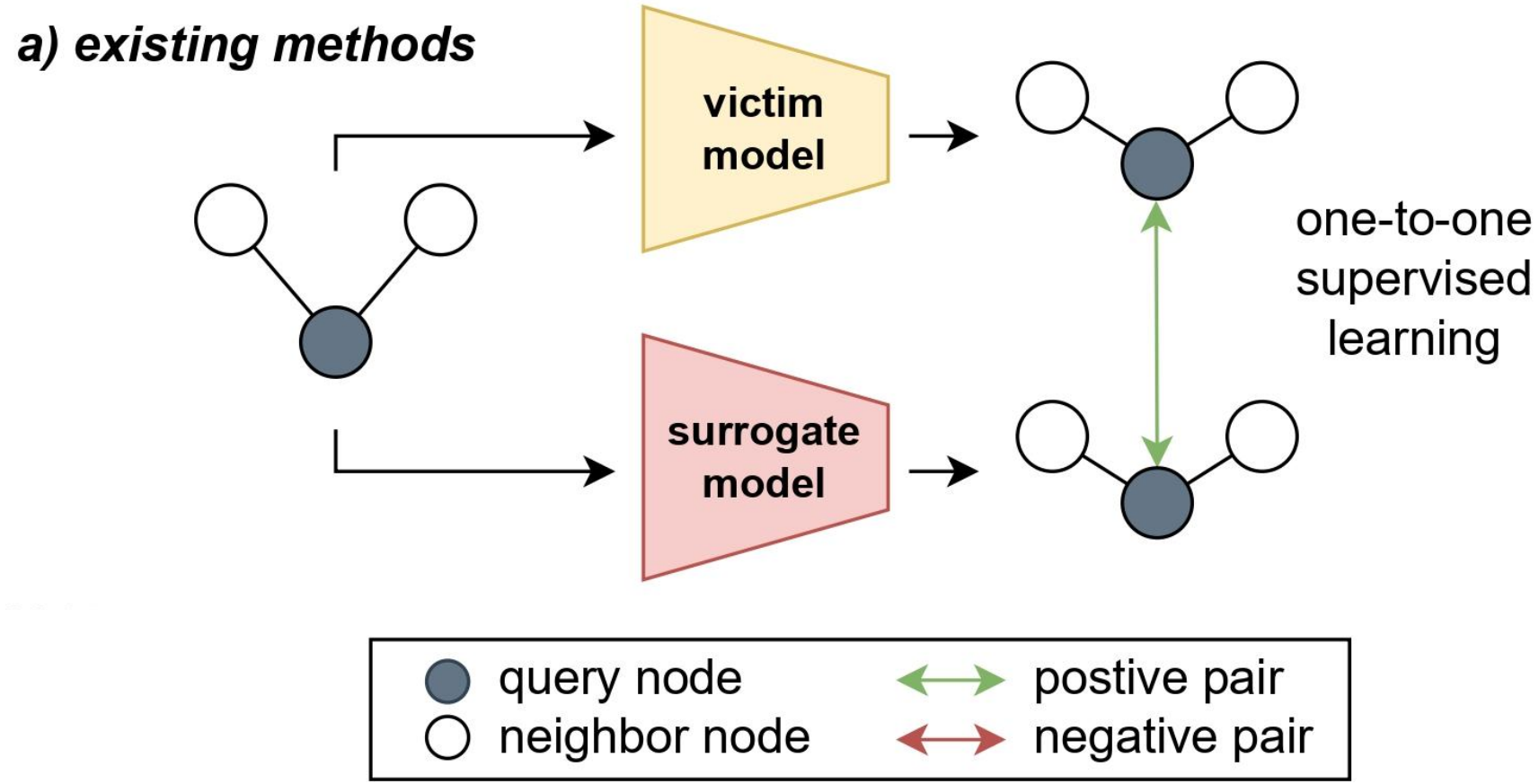
1. Pretext Task – The model solves a self-imposed challenge (e.g., predicting missing parts of an image or text).
2. Feature Learning – The model captures useful representations from data.
3. Fine-Tuning – The learned representations are used for downstream tasks (e.g., classification, detection).

What is contrastive learning?

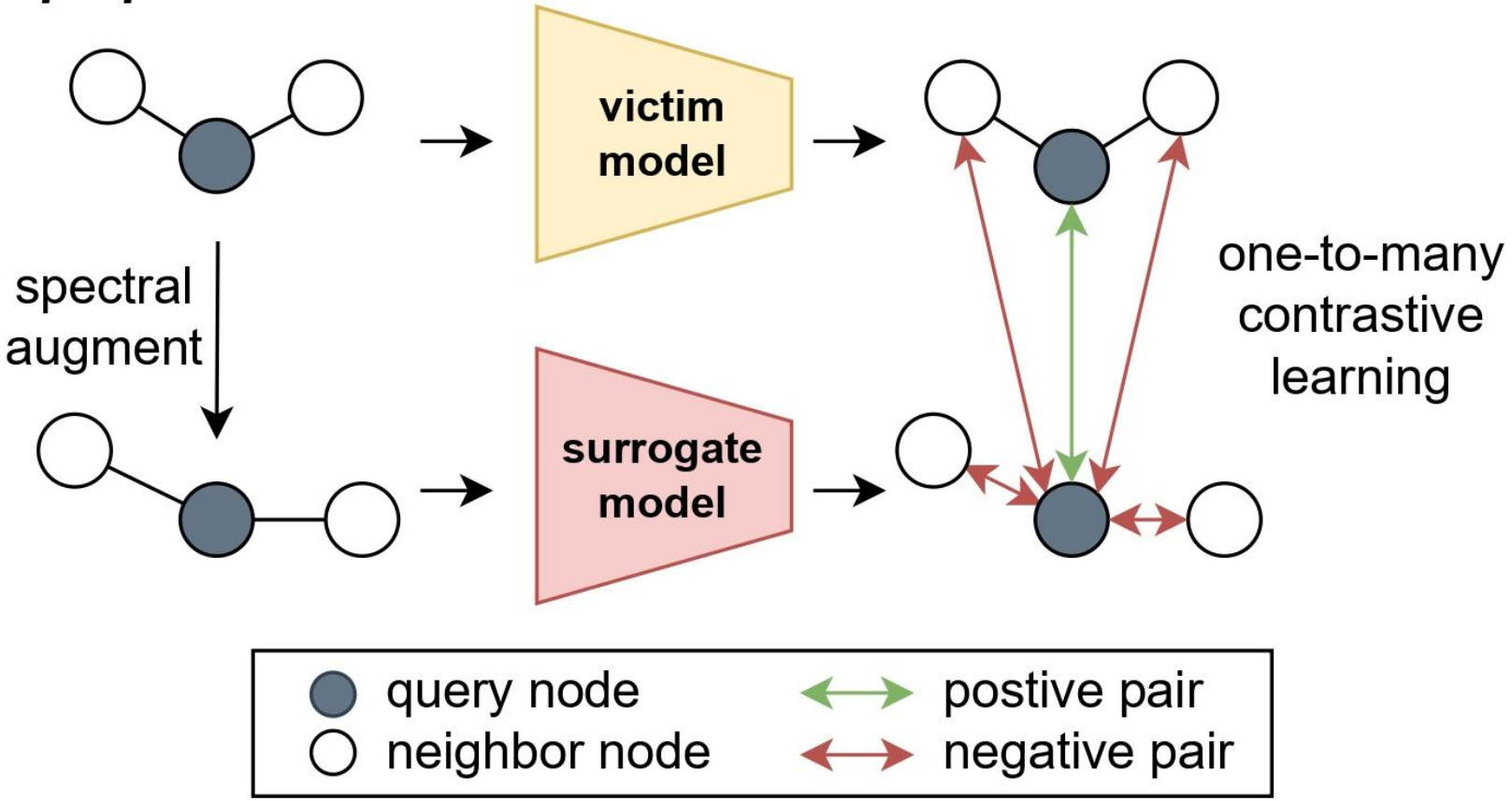
What is Contrastive Learning?

- A type of self-supervised learning where models learn by distinguishing similar and dissimilar examples.
- Focuses on bringing similar representations closer and pushing different ones apart in the embedding space.

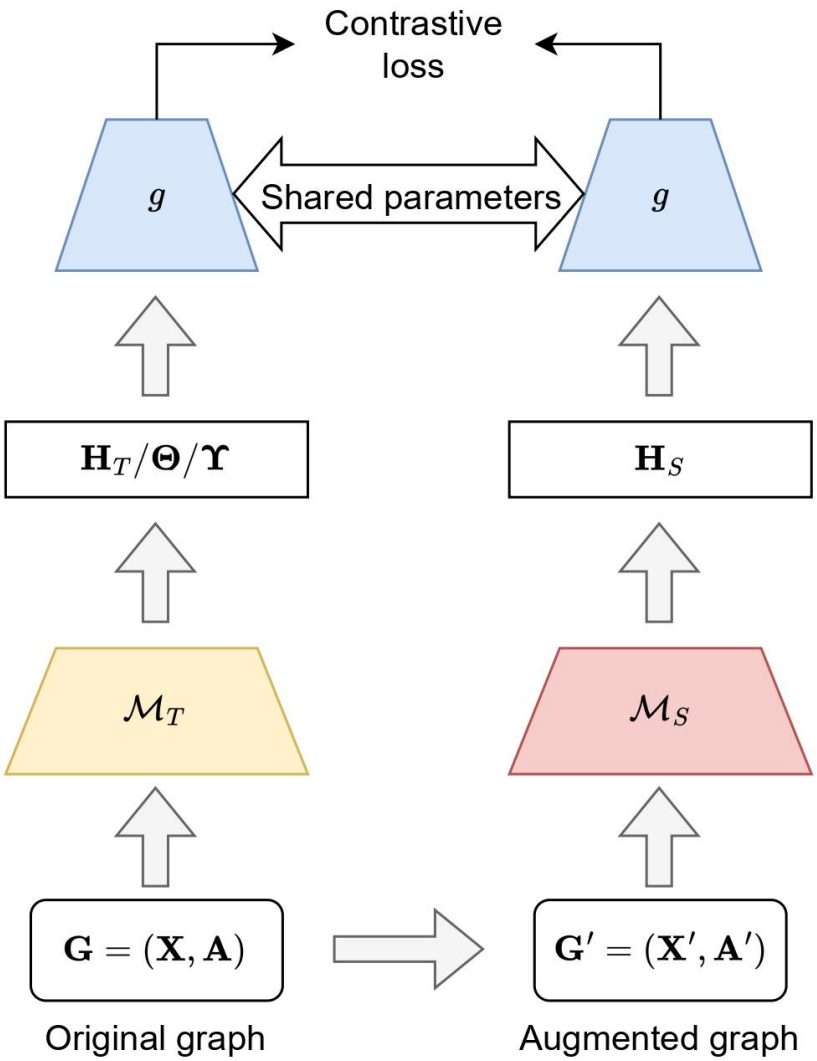

Positive: I^+
Anchor: I^a
Negative: I^-
Encoder
θ
$f^+ = \theta(I^+)$
$f^a = \theta(I^a)$
$f^- = \theta(I^-)$
Distance Function: δ
$\delta(f^a, f^+)$
Minimize
Distance Function: δ
$\delta(f^a, f^-)$
Maximize


a) existing methods
victim model
surrogate model
one-to-one supervised learning
query node
neighbor node
postive pair
negative pair

b) our proposed method

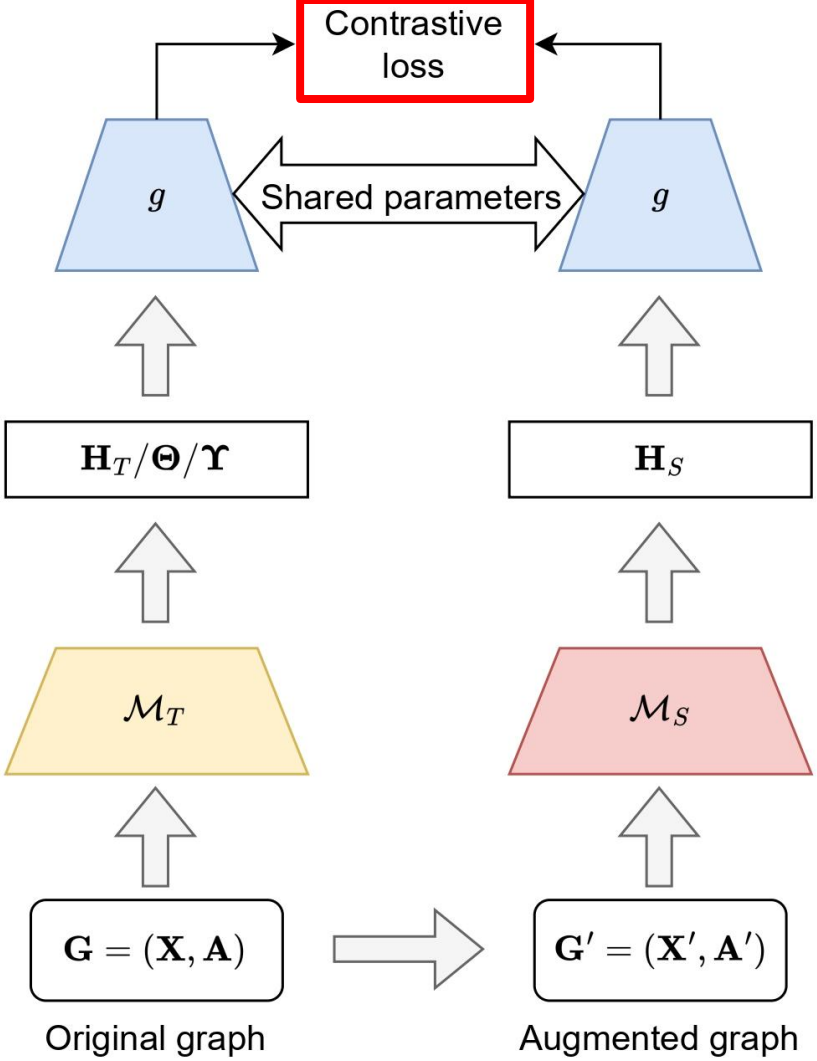


Our method


Contrastive loss
g
Shared parameters
g
$\mathbf{H}_T/\mathbf{\Theta}/\mathbf{\Upsilon}$
$\mathbf{H}_S$
$\mathcal{M}_T$
$\mathcal{M}_S$
$\mathbf{G} = (\mathbf{X}, \mathbf{A})$
$\mathbf{G}' = (\mathbf{X}', \mathbf{A}')$
Original graph
Augmented graph

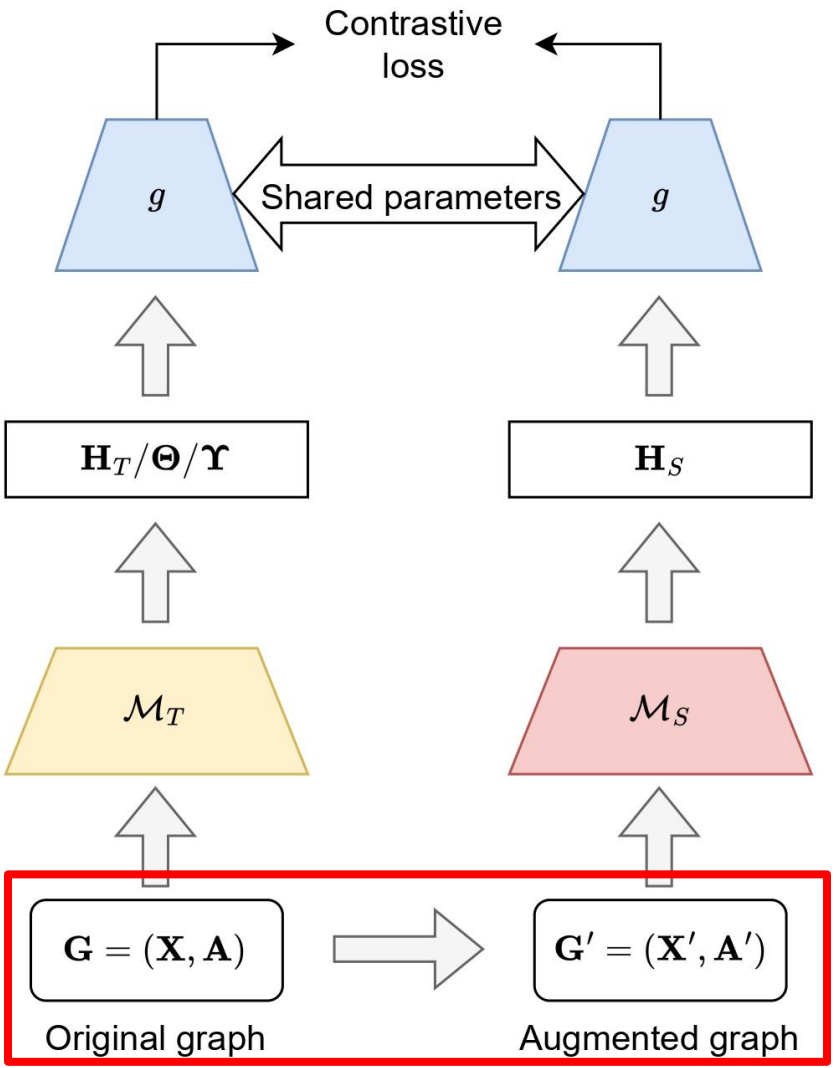
Our method

- The model learns to distinguish between pairs of node representations:
  - positive: same node from victim and surrogate model,
  - negative: different nodes.
- This minimizes the distance between positive representations, maximizing it for negative ones.
- This allows **efficient** use of **information from the victim model response**.



Our method

- Spectral augmentations:
  - Improve model stealing efficiency by making better use of information from the victim model results.
  - Allowing the surrogate model to **capture relevant information** about nodes while filtering out irrelevant noise.



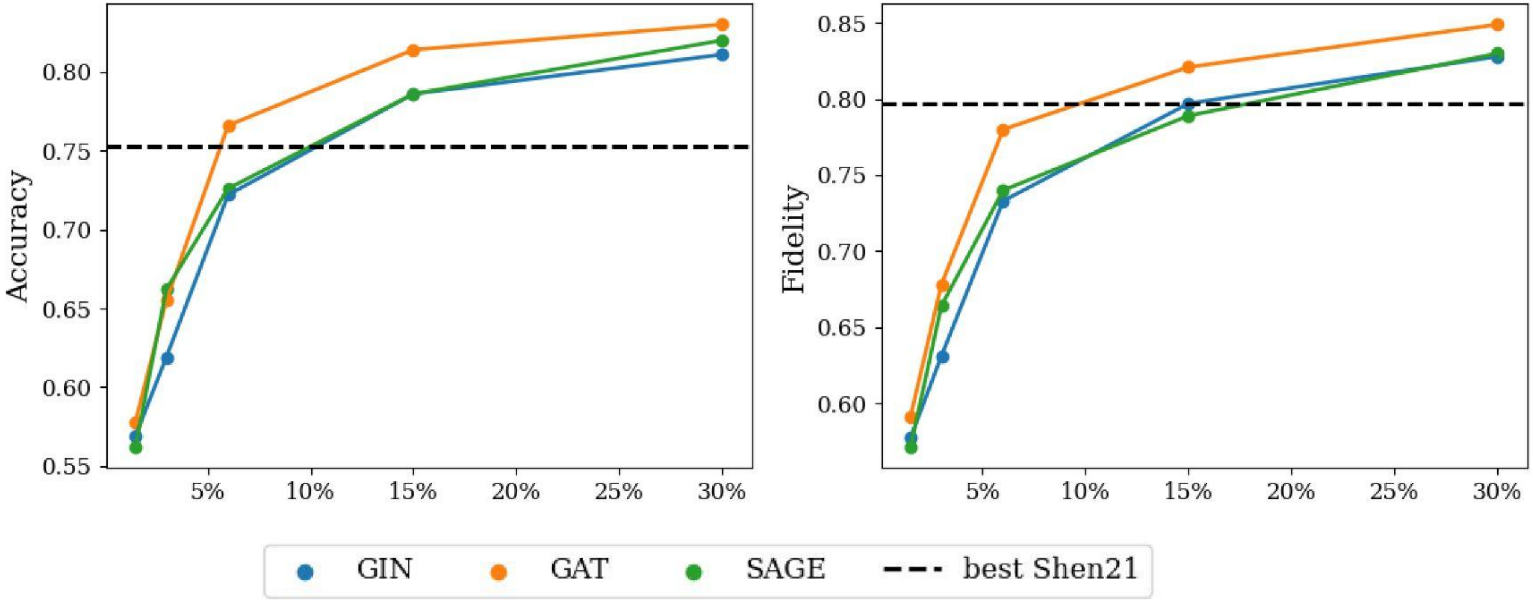
Our method is effective!

		Accuracy	Fidelity	F1 score
Prediction	Shen et al.	0.725±0.007	0.766±0.009	0.741±0.001
	ours	**0.827±0.003**	**0.861±0.003**	**0.827±0.005**
Projection	Shen et al.	0.704±0.012	0.733±0.015	0.754±0.005
	ours	**0.766±0.005**	**0.779±0.004**	**0.760±0.009**
Embedding	Shen et al.	0.730±0.004	0.771±0.007	0.782±0.001
	ours	**0.828±0.000**	**0.852±0.000**	**0.837±0.002**

Dataset: Citeseer, Target model: GIN, Surrogate model: GAT

Compared to the previous SOTA, **we achieve better metrics** when using the same number of queries (30% of the dataset)

Our method is efficient!



We achieve the previous SOTA performance **while needing ⅓ queries!**
(~10% of the dataset)

2nd Attack Strategy: Limited Access to Model

- What If We Have Data But Limited Access?
- When an attacker has access to a large dataset but is restricted in how many queries can be made to the model, the situation becomes trickier.

Assumptions

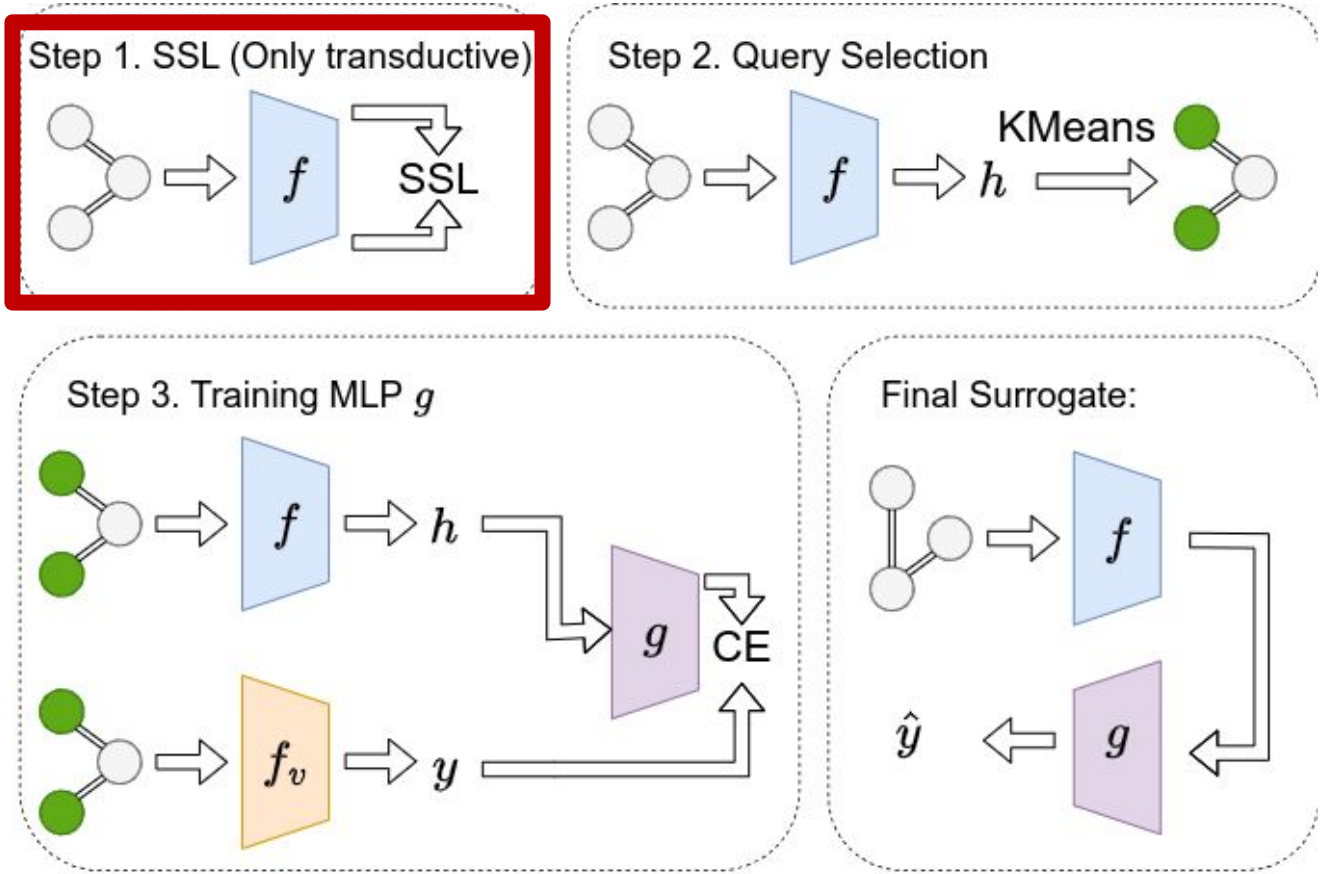
- Key Assumptions

- Plenty of Data: The attacker has more data than they can query, but no labels.
- Limited Queries: The attacker can make only a limited number of queries to the model.
- We assume that the target model returns only head predictions (the most restricted scenario)

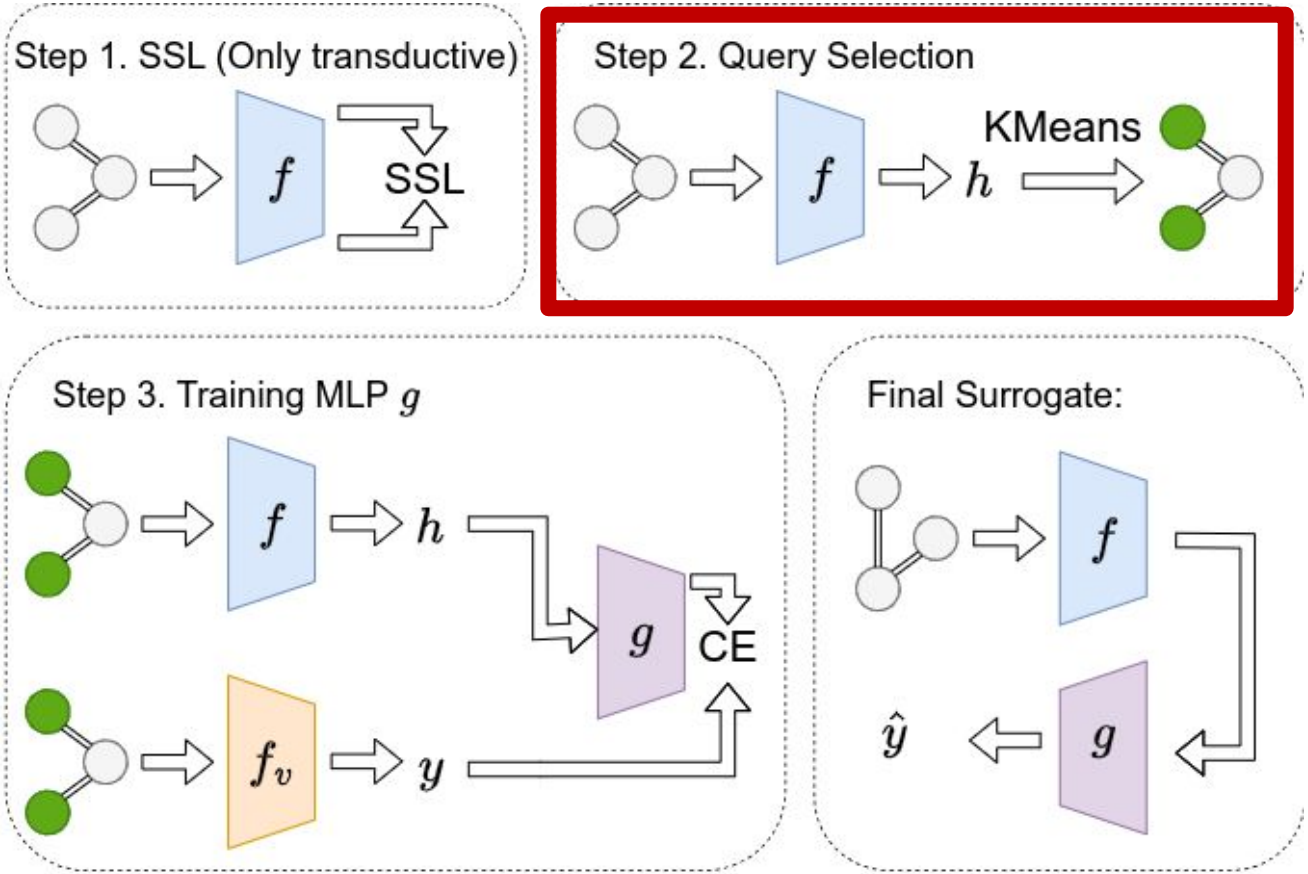
Questions

- What to do with all the available data?
- How to select a good query? Are some queries better than others?

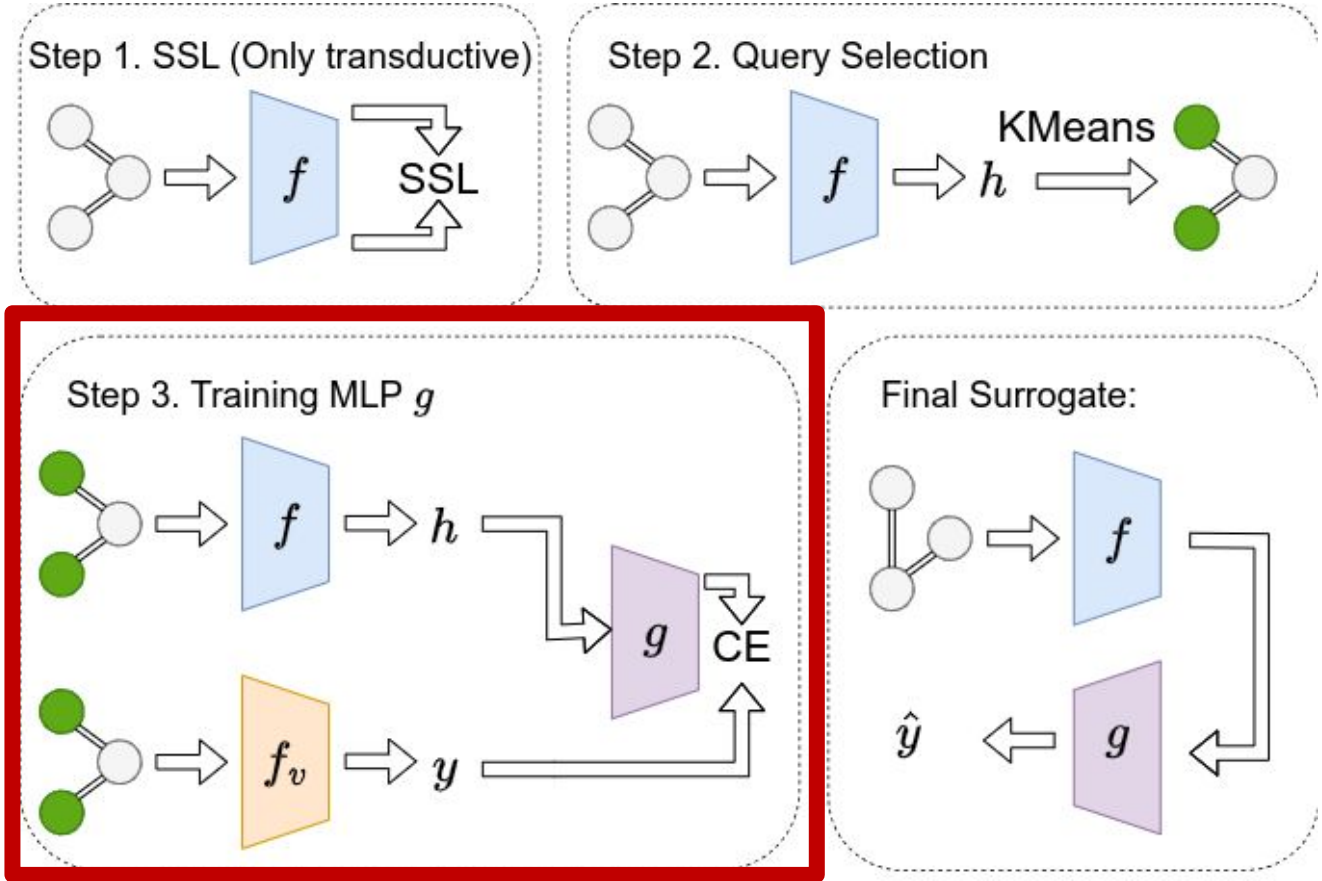
Train the encoder locally



Select query based on the limit



Train the head based on the target response



Dataset	Method	GCN Backbone		GIN Backbone	
		Accuracy	Fidelity	Accuracy	Fidelity
Cora	*E2E* [Wu *et al.*, 2021]	0.475±0.037	0.457±0.010	0.391±0.097	0.388±0.080
	SSL + Random	0.561±0.027	0.568±0.030	0.527±0.047	0.515±0.061
	[Zhuang *et al.*, 2024]	0.181±0.027	0.211±0.039	0.177±0.044	0.210±0.051
	SSL + Selection (ours)	**0.699±0.012**	**0.725±0.013**	**0.577±0.029**	**0.578±0.023**
Citeseer	*E2E* [Wu *et al.*, 2021]	0.372±0.061	0.411±0.075	0.378±0.031	0.401±0.055
	SSL + Random	0.513±0.051	0.576±0.055	0.633±0.021	0.565±0.035
	[Zhuang *et al.*, 2024]	0.221±0.033	0.231±0.038	0.233±0.028	0.260±0.040
	SSL + Selection (ours)	**0.663±0.019**	**0.724±0.023**	**0.656±0.010**	**0.715±0.012**
Pubmed	*E2E* [Wu *et al.*, 2021]	0.610±0.049	0.675±0.050	0.571±0.031	0.660±0.030
	SSL + Random	0.661±0.073	0.727±0.090	0.691±0.043	0.725±0.041
	[Zhuang *et al.*, 2024]	0.332±0.029	0.334±0.030	0.351±0.022	0.366±0.040
	SSL + Selection (ours)	**0.670±0.060**	**0.801±0.047**	**0.699±0.031**	**0.760±0.030**

Efficient Model-Stealing Attacks Against Inductive Graph Neural Networks

Marcin Podhajski, Jan Dubiński, Franziska Boenisch,
Adam Dziedzic, Agnieszka Pręgowska and Tomasz P. Michalak

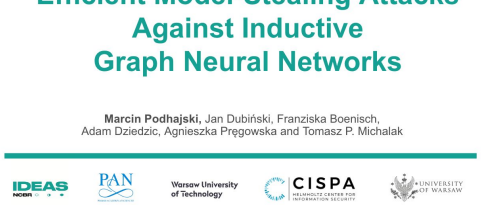
Explore the Full Paper and Code!

📄 Full Paper:

https://arxiv.org/abs/2405.12295

💻 Code Repository:

https://github.com/m-podhajski/EfficientGNNStealing



Questions?

marcin.podhajski@ideas-ncbr.pl